IFMT - Instituto Federal de Mato Grosso

Eng. da Computação



Criptografia de Chave Simétrica:

Fundamentos, algoritmos (DES/AES), modos de operação e implementação prática em Python

Prof. Dr. Ruy de Oliveira

Sumário

01 Fundamentos da criptografia simétrica

02 Evolução dos algoritmos: DES e AES

03 Modos de operação (ECB, CBC, CTR, GCM)

04 AES na prática com Python

05 Distribuição de chaves e boas práticas



SEÇÃO 01

Fundamentos da Criptografia Simétrica

Conceitos, analogias e a base matemática das cifras

FUNDAMENTOS

O que é criptografia de chave simétrica?

É o esquema criptográfico em que a mesma chave é usada tanto para cifrar quanto para decifrar a mensagem. Quem cifra e quem decifra precisam compartilhar o mesmo segredo previamente.

Alto desempenho

Operações rápidas, ideais para grandes volumes de dados.

Segurança pela chave

Toda a segurança depende do sigilo da chave compartilhada.

Mesma chave, duas vias

Cifrar(M, K) ⟶ C e Decifrar(C, K) ⟶ M



Plaintext → Texto em claro ou texto claro ou texto original ou texto puro
Ciphertext → Texto cifrado
Encryption → Cifragem ou Criptografia
Decryption → Decifragem ou Descriptografia
Key → Chave

FUNDAMENTOS

Uma analogia simples: o cofre físico

Imagine um cofre físico com apenas um tipo de chave. Quem guarda um documento dentro (cifra) e quem depois o retira (decifra) precisam ter cópias idênticas da mesma chave.



1. Guardar

Alice tranca o documento dentro do cofre usando a chave K.



2. Transportar

O cofre trancado pode viajar por um canal inseguro sem risco.



3. Abrir

Bob usa a mesma chave K para destrancar o cofre e ler o conteúdo.

O grande desafio: como entregar a chave do cofre ao destinatário com segurança? (voltaremos a esse ponto)

FUNDAMENTOS

Cifras de bloco, cifras de fluxo e a base: XOR

Cifra de bloco

Divide os dados em blocos de tamanho fixo (ex.: 128 bits no AES) e cifra cada bloco.

Cifra de fluxo

Combina cada bit/byte do texto claro com um fluxo de bits pseudoaleatório, geralmente via XOR.

A operação XOR (⊕)

XOR é reversível: aplicar a mesma chave duas vezes recupera o valor original. É a base de muitas cifras simétricas.

A	B	A ⊕ B
0	0	**0**
0	1	**1**
1	0	**1**
1	1	**0**

Comutativa: $a \oplus b = b \oplus a$
Associativa: $(a \oplus b) \oplus c = a \oplus (b \oplus c)$
Elemento neutro: $a \oplus 0 = a$
Autoinversa: $a \oplus a = 0$

$$A \oplus B \oplus B = A$$

POR QUE ISSO IMPORTA PARA CRIPTOGRAFIA

Se $C = M \oplus K$, então $(C \oplus K) = (M \oplus K) \oplus K = M \oplus (K \oplus K) = M \oplus 0 = M$

A propriedade autoinversa é o que permite usar a MESMA operação para cifrar e decifrar.

FUNDAMENTOS · PYTHON

Cifra XOR em Python

```python
# Cifra simétrica mais simples possível
def xor_cifra(dados: bytes, chave: bytes) -> bytes:
    """Cifra ou decifra dados via XOR com a chave"""
    return bytes(
        b ^ chave[i % len(chave)]
        for i, b in enumerate(dados)
    )

mensagem = b"Ataque ao amanhecer"
chave = b"segredo123"

cifrado = xor_cifra(mensagem, chave)
decifrado = xor_cifra(cifrado, chave)

print ("cifrado:", cifrado, "\ndecifrado:", decifrado)
print(decifrado == mensagem)
# Saída: True
```

A mesma função cifra e decifra: XOR aplicado duas vezes com a mesma chave devolve o valor original. Simples, mas repetir a chave a torna previsível — por isso cifras reais (AES) são bem mais elaboradas.



SEÇÃO 02

Evolução dos Algoritmos

Do DES ao AES: como os padrões evoluíram

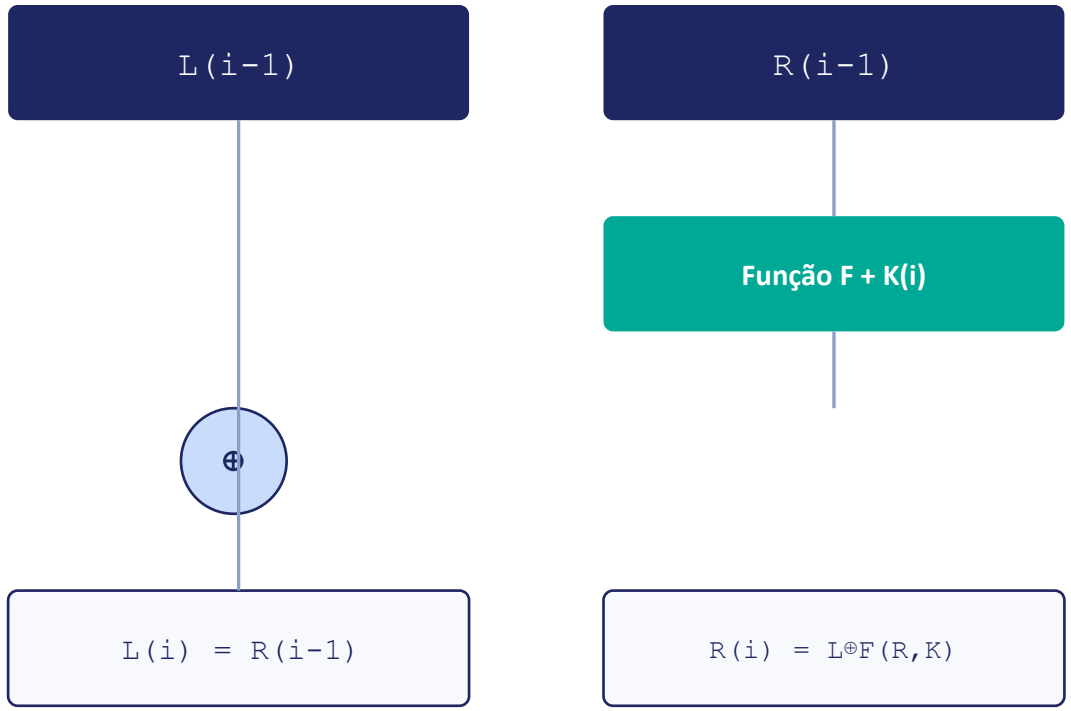
EVOLUÇÃO DOS ALGORITMOS

DES: o padrão pioneiro (1977)

O DES (Data Encryption Standard) foi o primeiro padrão de cifra de bloco amplamente adotado pela indústria, publicado pelo NIST em 1977. Usa blocos de 64 bits e uma estrutura chamada rede de Feistel.

Tamanho do bloco	**64 bits**
Tamanho da chave	**56 bits efetivos**
Estrutura	**Rede de Feistel, 16 rounds**

Estrutura de Feistel (uma rodada)



Repetida 16 vezes com sub-chaves K(i) diferentes, derivadas da chave de 56 bits.

DES: o padrão pioneiro (1977)

O **DES (Data Encryption Standard)** é um método clássico usado para embaralhar informações, de modo que apenas quem possui uma chave secreta consiga lê-las. Pense nele como um quebra-cabeça digital complexo.

Simplificadamente, o DES funciona da seguind maneira:

- **Chave Simétrica:** O DES usa uma única chave secreta (de 56 bits) tanto para trancar (encriptar) quanto para destrancar (decriptar) a mensagem.
- **Cifra de Bloco:** Em vez de encriptar o texto letra por letra, o DES processa **blocos de 64 bits**, um por vez
- **Rede de Feistel (16 Rodadas):** Para embaralhar um bloco, o DES o divide em duas metades (Esquerda e Direita) e as submete a **16 rodadas (rounds) idênticas** de mistura. Em cada rodada:
  - A metade Direita é combinada com uma parte da chave secreta usando funções matemáticas
  - O resultado é combinado com a metade Esquerda através de uma operação **XOR**
  - As duas metades trocam de posição para a próxima rodada
- **Substituição e Permutação:** Durante cada rodada, o DES usa **S-Boxes** (Caixas de Substituição) para trocar bits por outros diferentes (adicionando *confusão*) e **P-Boxes** (Caixas de Permutação) para reordenar os bits (adicionando *difusão*)
- **Resultado Final:** Após todas as 16 rodadas e um embaralhamento final, o bloco legível original se transforma em um **texto cifrado** (texto encriptado) irreconhecível de 64 bits

Como sua chave de 56 bits hoje é considerada muito pequena e fácil de ser quebrada por força bruta em computadores modernos, o DES foi substituído por padrões mais rápidos e muito mais seguros, como o **AES** (Advanced Encryption Standard)

DES: o padrão pioneiro (1977)

Primeiro padrão público de criptografia de bloco, base da criptografia moderna.

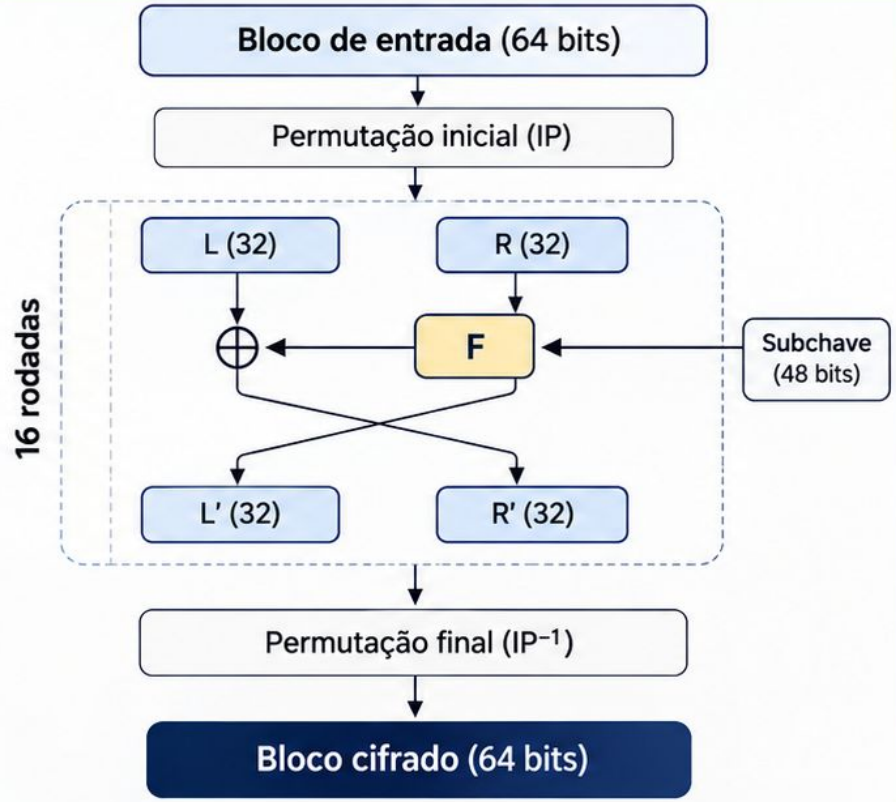
Resumo

- Cifra de bloco
- Bloco: 64 bits
- Chave: 56 bits (64 bits com 8 de paridade)
- Estrutura: Rede de Feistel
- 16 rodadas

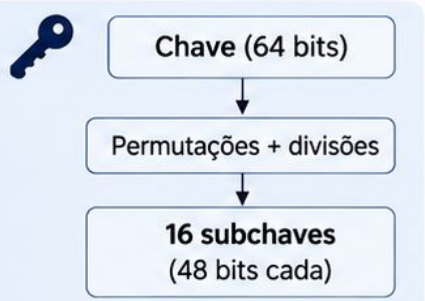
Por que foi importante?

- ✓ Primeiro padrão federal (NBS)
- ✓ Larga adoção mundial
- ✓ Inspirou algoritmos futuros (AES, 3DES, etc.)

Como funciona — Cifração



Geração das subchaves



Linha do tempo



Observação

Com o avanço da computação, a chave de 56 bits tornou-se pequena. Hoje, o DES não é mais seguro.

EVOLUÇÃO DOS ALGORITMOS

AES: o padrão atual (2001)

Após um concurso público, o NIST escolheu o algoritmo Rijndael como novo padrão em 2001: o AES (Advanced Encryption Standard). Ele substituiu o DES e hoje protege desde HTTPS até discos criptografados.

128 bits

Tamanho do bloco (fixo)

128 / 192 / 256 bits

Tamanhos de chave suportados

10 / 12 / 14

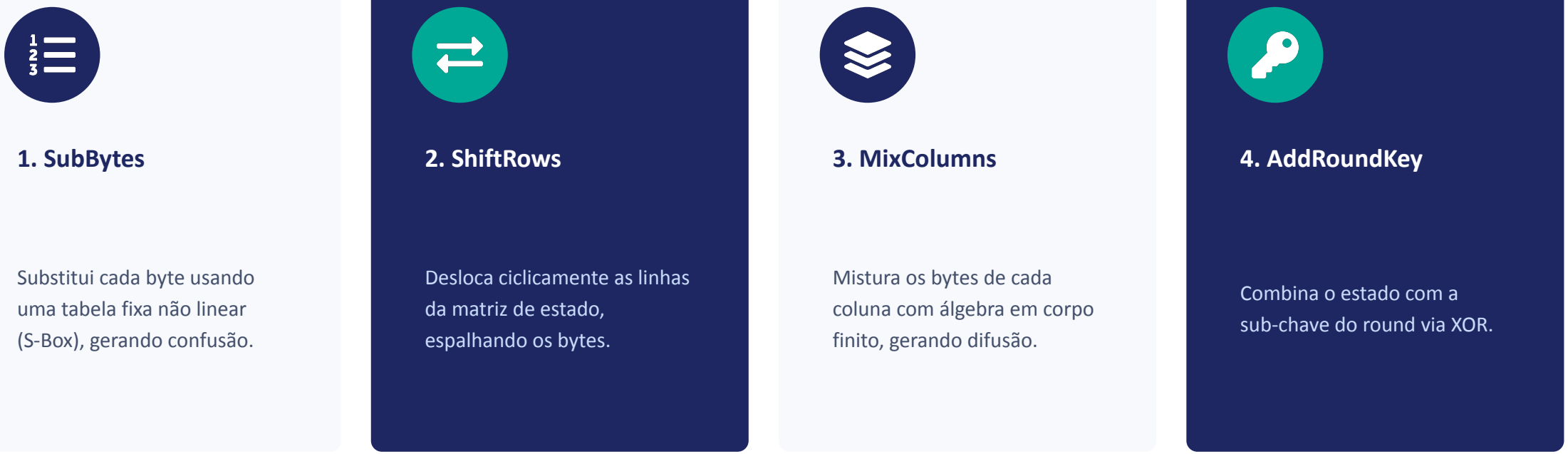
Número de rounds (conforme a chave)

Por que o AES venceu?

- Resistente a todos os ataques criptoanalíticos conhecidos até hoje
- Eficiente em software e hardware (inclusive instruções de CPU dedicadas — AES-NI)
- Sem patentes: livre para qualquer implementação

Estrutura interna de um round do AES

Cada round do AES aplica quatro transformações em sequência sobre o estado (uma matriz de bytes 4×4):

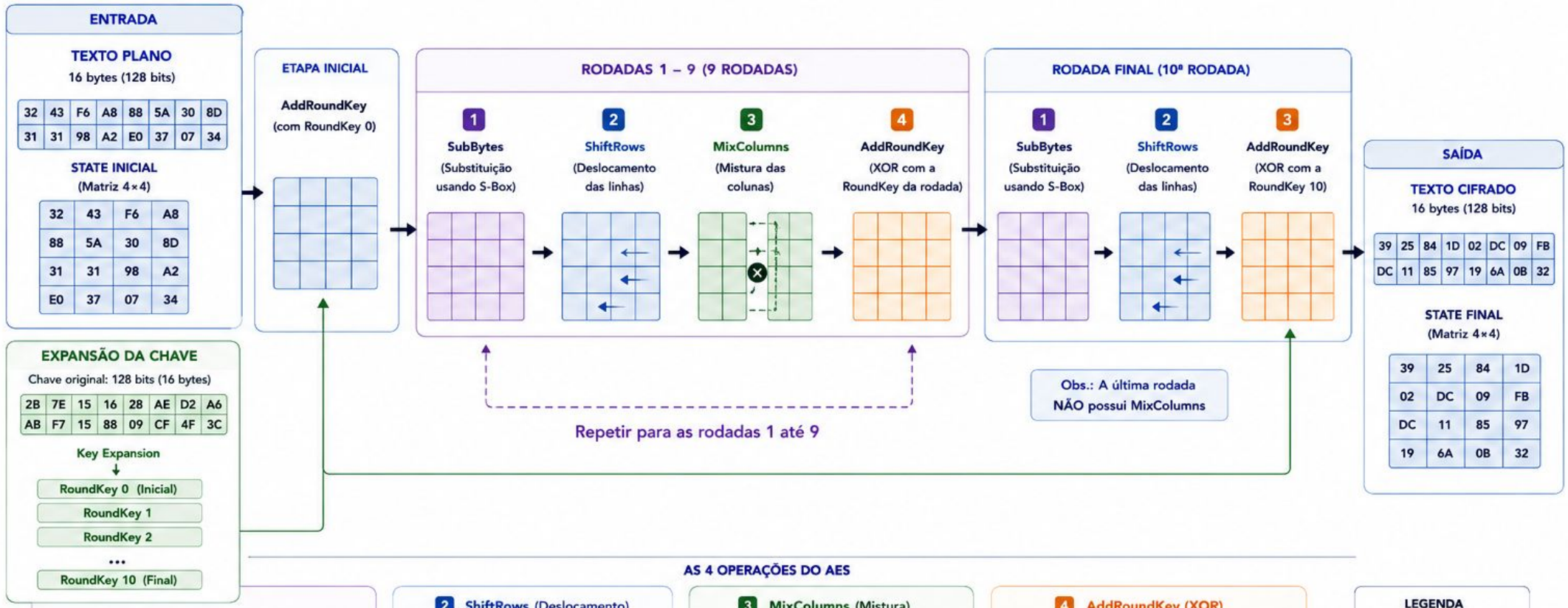


O último round omite o MixColumns. O número total de rounds depende do tamanho da chave: 10 (128 bits), 12 (192) ou 14 (256).

Confusão: Relação complexa entre a chave e o texto cifrado (ex.: *SubBytes* no AES), garantindo que mudar um bit da chave altere vários bits do texto cifrado e impeça a dedução da chave.
Difusão: Espalhamento da estrutura do texto claro por todo o texto cifrado (ex.: *ShiftRows* e *MixColumns* no AES), garantindo que alterar um único bit da mensagem original mude cerca de metade dos bits do texto cifrado.

AES-128 – FLUXO COMPLETO

Criptografia de um bloco de 128 bits (16 bytes)



AS 4 OPERAÇÕES DO AES

1 SubBytes (Substituição)

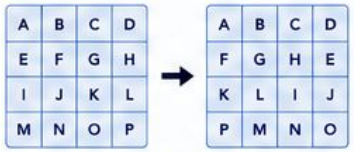
Cada byte do State é substituído por outro usando a S-Box.



Exemplo: 53 → ED

2 ShiftRows (Deslocamento)

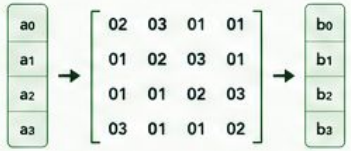
Cada linha da matriz é deslocada ciclicamente para a esquerda.



Deslocamentos: 0, 1, 2 e 3 posições

3 MixColumns (Mistura)

Cada coluna é transformada usando operações matemáticas em GF(2^8).



Multiplicações e somas em GF(2^8)

4 AddRoundKey (XOR)

Cada byte do State é combinado (XOR) com a chave da rodada.



a⊕ko significa a XOR ko

LEGENDA

- Entrada / Saída
- SubBytes
- ShiftRows
- MixColumns
- AddRoundKey
- Uso da chave (RoundKey)

PRINCÍPIOS DE SEGURANÇA

CONFUSÃO: S-Box e AddRoundKey dificultam a relação entre chave e texto cifrado.

DIFUSÃO: ShiftRows e MixColumns espalham as alterações por todo o State (efeito avalanche).

O AES-128 possui 10 rodadas.
AES-192 possui 12 rodadas.
AES-256 possui 14 rodadas.



SEÇÃO 03

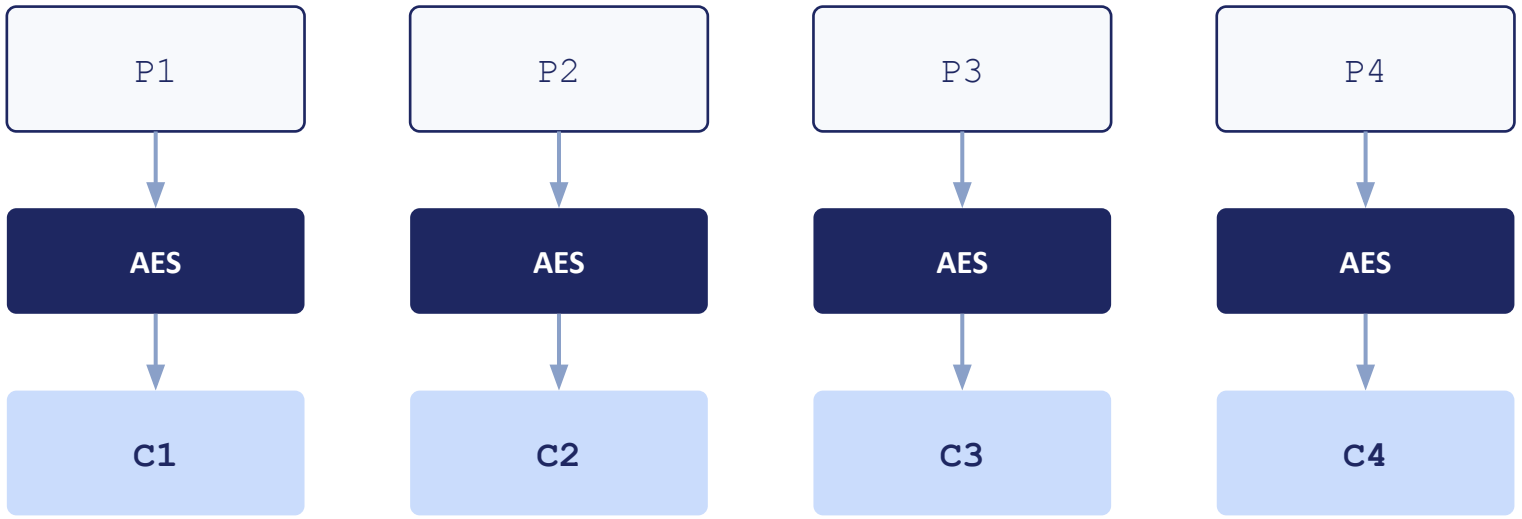
Modos de Operação

Como blocos individuais viram um fluxo cifrado seguro

MODOS DE OPERAÇÃO

ECB (Electronic Codebook): simplicidade perigosa

Cada bloco de texto claro é cifrado de forma independente, sempre com a mesma chave — sem qualquer ligação entre blocos.

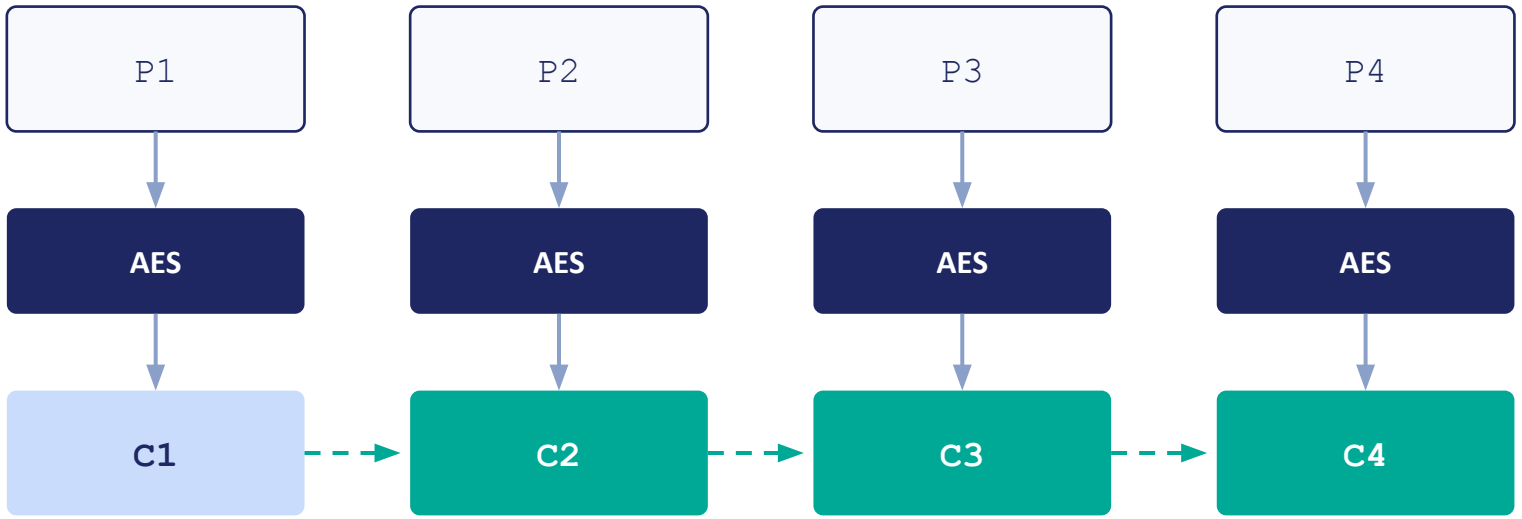


Blocos idênticos de entrada geram blocos idênticos de saída: padrões visuais e estruturais vazam. Não deve ser usado para dados estruturados ou imagens.

MODOS DE OPERAÇÃO

CBC (Cipher Block Chaining): encadeamento com IV

Antes de cifrar, cada bloco de texto claro é combinado (XOR) com o bloco cifrado anterior. O primeiro bloco usa um IV (vetor de inicialização) aleatório.



Elimina o padrão do ECB, mas exige um IV único e imprevisível a cada mensagem — reutilizar o IV compromete a segurança.

Vetor de Inicialização: Bloco de 16 bytes que é usado como "C_0" — o bloco cifrado "fictício" que precede o primeiro bloco real.

MODOS DE OPERAÇÃO

CTR e GCM: paralelismo e autenticação



CTR — Counter Mode

Transforma o AES em uma cifra de fluxo: cifra um contador crescente e faz XOR do resultado com o texto claro.

- Blocos podem ser cifrados em paralelo
- Não exige padding (pode cifrar qualquer tamanho)
- Não fornece autenticação por si só



GCM — Galois/Counter Mode

Combina o CTR com um MAC (GMAC) calculado sobre o texto cifrado: cifra E autentica em uma única passada.

- Detecta qualquer alteração no texto cifrado
- É o padrão recomendado hoje (TLS 1.3, por ex.)
- Nunca reutilize o mesmo par (chave, nonce)

Modos de Operação do AES

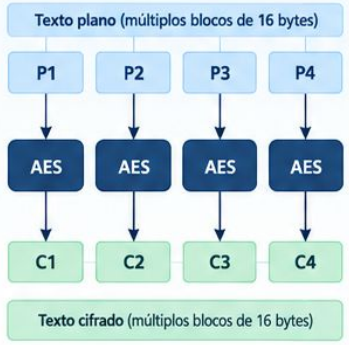
O AES é um cifrador de blocos de 128 bits. Ele criptografa dados em blocos de 16 bytes (128 bits). Os modos de operação definem como múltiplos blocos são processados para criptografar mensagens maiores e fornecem propriedades de segurança adicionais.

Tamanho do bloco e da chave do AES

Tamanho do bloco	128 bits (16 bytes)
Tamanhos de chave	128 / 192 / 256 bits (16 / 24 / 32 bytes)

ECB
Cifrário de Código Eletrónico

Cada bloco é criptografado de forma independente. Blocos idênticos de texto plano produzem blocos idênticos de cifrado.



Como funciona

1. Divide o texto plano em blocos de 16 bytes.
2. Criptografa cada bloco com a mesma chave AES.
3. Concatena os blocos cifrados.

Principais propriedades

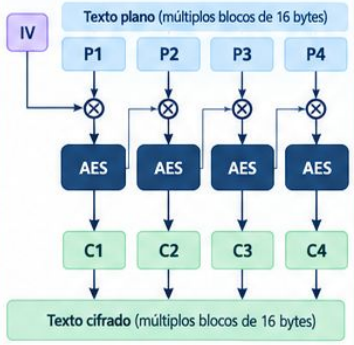
- Simples e rápido.
- Não encadeia os blocos.
- Blocos idênticos no texto plano geram blocos idênticos no cifrado (vazamento de padrões).

Casos de uso típicos

- Raramente usado sozinho.
- Pode ser usado em sistemas legados ou em cenários de compatibilidade.

CBC
Encadeamento de Blocos de Cifra

Cada bloco é combinado (XOR) com o bloco cifrado anterior antes de ser criptografado. O primeiro bloco usa um vetor de inicialização (IV).



Como funciona

1. Faz XOR do primeiro bloco com o IV.
2. Criptografa com AES.
3. Para cada próximo bloco, faz XOR com o bloco cifrado anterior, depois criptografa.
4. Concatena os blocos cifrados.

Principais propriedades

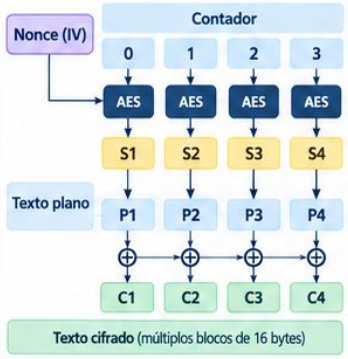
- Esconde padrões no texto plano.
- Requer um vetor de inicialização (IV) único.
- A criptografia e a descriptografia são sequenciais (não são paralelizáveis facilmente).

Casos de uso típicos

- Amplamente usado na prática (ex.: TLS 1.2, IPsec).
- Adequado para criptografia de uso geral e de fluxos de dados.

CTR
Modo de Contador

Transforma o AES em um cifrador de fluxo usando um contador e o XOR com o texto plano. Os blocos podem ser processados em paralelo.



Como funciona

1. Combina o IV (nonce) com o valor do contador.
2. Criptografa com AES para gerar uma sequência de chaves (keystream).
3. Faz XOR do keystream com o texto plano.
4. Incrementa o contador e repete.

Principais propriedades

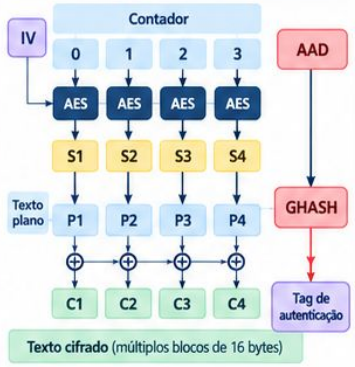
- Não encadeia blocos.
- Pode ser paralelizado (rápido).
- Requer um nonce/contador único por mensagem.
- Não fornece autenticação (deve ser combinado com um modo como GCM).

Casos de uso típicos

- Transmissão de dados em tempo real.
- Aplicações que precisam de alta performance.
- Muito usado com autenticação (ex.: GCM).

GCM
Modo Galois/Contador

Combina o CTR para confidencialidade com o GHASH (campo de Galois) para autenticação. Fornece confidencialidade e integridade.



Como funciona

1. Gera o keystream usando o CTR (como no CTR).
2. Faz XOR do keystream com o texto plano.
3. Calcula a tag de autenticação usando GHASH (sobre o texto cifrado e o AAD opcional).
4. Produz o texto cifrado + a tag de autenticação.

Principais propriedades

- Confidencialidade (como o CTR).
- Integridade e autenticidade (via GHASH).
- Paralelizável (rápido).
- Suportado amplamente (ex.: TLS 1.3, SSH, QUIC).

Casos de uso típicos

- Comunicação segura (TLS 1.3, QUIC).
- Criptografia de arquivos e disco.
- Aplicações que exigem confidencialidade e integridade.

Em resumo: o **AES sempre usa blocos de 16 bytes.** O **modo de operação** define como esses blocos são combinados, como o dado é encadeado (ou não) e se a **autenticação é fornecida.**

- O texto plano provê confidencialidade, e o a Tag prov&e integridade e autencidade

Modos de Operação do AES

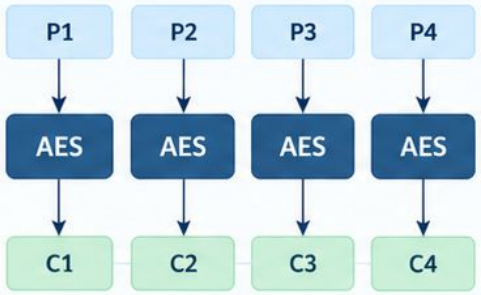
O AES é um cifrador de blocos de 128 bits. Os modos de operação definem como múltiplos blocos são processados para criptografar mensagens maiores.

Tamanho do bloco
128 bits (16 bytes)

Tamanhos de chave
128 / 192 / 256 bits
(16 / 24 / 32 bytes)

ECB
Cifrário de Código Eletrônico

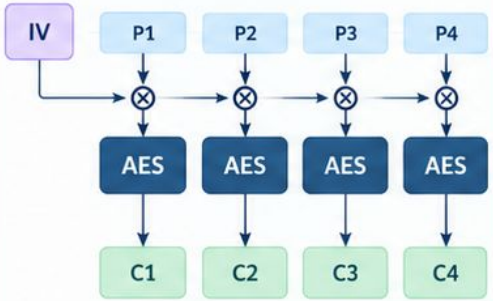
Cada bloco é criptografado de forma independente.



Destaques
- Simples e rápido
- Não usa vetor de inicialização
- Padrão fraco em segurança

CBC
Encadeamento de Blocos de Cifra

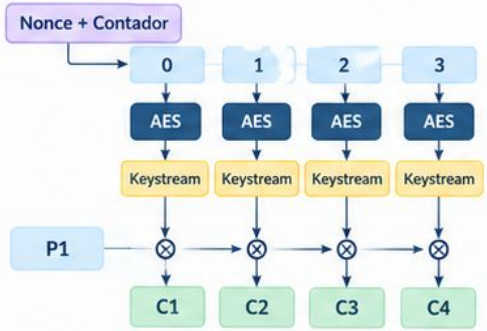
Cada bloco é combinado com o bloco anterior (via XOR) antes de ser cifrado.



Destaques
- Mais seguro que o ECB
- Requer um vetor de inicialização (IV)
- Não permite paralelização completa

CTR
Modo de Contador

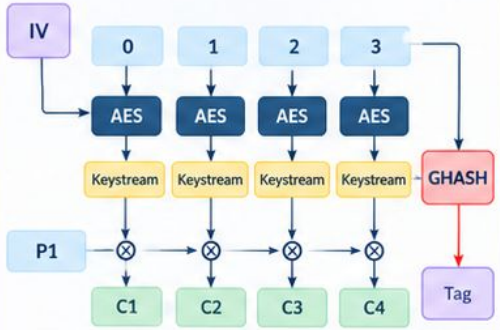
Usa um contador para gerar um fluxo de chaves (keystream) e faz XOR com o texto plano.



Destaques
- Paralelização (mais rápido)
- Não precisa de padding
- Boa para transmissão e alta performance

GCM
Modo Galois/Counter

Combina o CTR para confidencialidade com um código de autenticação (GHASH) para integridade.



Destaques
- Confidencialidade (como CTR)
- Integridade e autenticação
- Suporte a dados em paralelo

Em resumo: o AES sempre usa blocos de **16 bytes.** O modo de operação define como esses blocos são combinados, como o dado é encadeado (ou não) e se a autenticação é fornecida.

SEÇÃO 04

AES na Prática com Python

Da teoria ao código com a biblioteca cryptography

AES NA PRÁTICA · PYTHON

Fernet — criptografia simétrica de alto nível — 1/2 · cifrando

python

```python
from cryptography.fernet import Fernet

# 1. Gerar uma chave simétrica aleatória
chave = Fernet.generate_key()
f = Fernet(chave)

# 2. Cifrar a mensagem
mensagem = b"Dados confidenciais do aluno"
token = f.encrypt(mensagem)

print(token)
# Saída: b'gAAAAABk...' (token cifrado e autenticado)
```

generate_key() cria uma chave AES-128 codificada em base64 — guarde-a com o mesmo cuidado de uma senha.

Fernet — criptografia simétrica de alto nível — 1/2 · cifrando

python

generate_key() cria uma chave AES-128 codificada em base64 — guarde-a com o mesmo cuidado de uma senha.

Fernet — criptografia simétrica de alto nível — 2/2 · decifrando

python

```python
# 3. Decifrar o token com a mesma chave
mensagem_original = f.decrypt(token)

print(mensagem_original)
# Saída: b'Dados confidenciais do aluno'


# Por baixo dos panos, o Fernet usa:
#   - AES-128 em modo CBC (confidencialidade)
#   - HMAC-SHA256 (integridade e autenticidade)
#   - IV e timestamp embutidos no próprio token
```

Se o token for alterado ou a chave estiver errada, decrypt() lança InvalidToken — o Fernet já cuida da verificação para você.

AES NA PRÁTICA · PYTHON

AES-GCM — controle de baixo nível — 1/2 · cifrando

python

```python
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
import os

chave = AESGCM.generate_key(bit_length=256)
aesgcm = AESGCM(chave)

# Nonce de 12 bytes: NUNCA reutilizar com a mesma chave
nonce = os.urandom(12)

dados = b"Nota do aluno: 9.5"
associados = b"cabecalho-publico"  # autenticado, nao cifrado

cifrado = aesgcm.encrypt(nonce, dados, associados)
print(cifrado.hex())
```

'associados' (AAD) é autenticado mas permanece em texto claro — útil para cabeçalhos que precisam ser lidos sem decifrar tudo.

AES-GCM — controle de baixo nível — 2/2 · decifrando

python

```python
# Decifrando (e autenticando) os dados
texto_claro = aesgcm.decrypt(
    nonce, cifrado, associados
)

print(texto_claro)
# Saída: b'Nota do aluno: 9.5'


# Se 'cifrado' ou 'associados' forem alterados,
# decrypt() lança cryptography.exceptions.InvalidTag:
# a integridade foi violada.
```

guarde nonce, associados e o texto cifrado juntos — todos são necessários para decifrar corretamente.

DISTRIBUIÇÃO DE CHAVES

O desafio central: como compartilhar a chave?

A criptografia simétrica pressupõe que ambas as partes já possuem a mesma chave. Mas como entregá-la com segurança pela primeira vez, especialmente por um canal inseguro como a internet?



Canal seguro à parte

Combinar a chave pessoalmente ou por outro meio confiável — inviável em escala.



Criptografia assimétrica

Usar um par de chaves pública/privada só para negociar a chave simétrica (ex.: TLS).



Acordo de chaves

Protocolos como Diffie-Hellman permitem derivar um segredo comum sem transmiti-lo.

PANORAMA GERAL

Criptografia simétrica vs. assimétrica

Critério	Simétrica	Assimétrica
Número de chaves	Uma única chave compartilhada	Par de chaves: pública e privada
Velocidade	Muito rápida	Consideravelmente mais lenta
Tamanho de dados	Ideal para grandes volumes	Ideal para pequenos volumes (ex.: chaves)
Principal desafio	Distribuir a chave com segurança	Custo computacional mais alto
Exemplos	AES, ChaCha20, (DES)	RSA, ECC, Diffie-Hellman

Na prática, os dois mundos se combinam: TLS usa criptografia assimétrica para negociar uma chave simétrica, que depois cifra o tráfego de dados.

RECOMENDAÇÕES

Boas práticas de uso

Prefira AES-256 ou ChaCha20-Poly1305

Use tamanhos de chave e algoritmos modernos, evitando DES e chaves curtas.

Nunca reutilize IV/nonce com a mesma chave

A reutilização pode expor o texto claro ou quebrar completamente a segurança.

Prefira modos autenticados (AEAD)

GCM ou ChaCha20-Poly1305 garantem confidencialidade e integridade juntas.

Derive chaves a partir de senhas com KDF

Use PBKDF2, scrypt ou Argon2 — nunca a senha diretamente como chave.

Gerencie chaves em um cofre dedicado

Serviços como KMS/HSM evitam guardar segredos em código-fonte.

Rotacione chaves periodicamente

Limite o impacto de um eventual vazamento de chave.

RESUMO

Conclusões

- Criptografia simétrica usa a mesma chave para cifrar e decifrar — é rápida e ideal para grandes volumes de dados.
- O AES substituiu o DES como padrão e é considerado seguro quando usado com chaves de 128 bits ou mais.
- O modo de operação importa tanto quanto o algoritmo: prefira sempre modos autenticados como o GCM.
- O maior desafio da criptografia simétrica não é o algoritmo, e sim a distribuição segura da chave.

MATERIAL DE APOIO

Referências

- STALLINGS, W.; BROWN, L. Computer Security: Principles and Practice. 5th ed. Pearson, 2023. Capítulos sobre criptografia simétrica.
- NIST FIPS 197 — Advanced Encryption Standard (AES). National Institute of Standards and Technology.
- NIST SP 800-38A/D — Recommendation for Block Cipher Modes of Operation (incluindo GCM).
- Documentação oficial do pacote Python cryptography (pyca/cryptography).
- ANDERSON, R. Security Engineering. 3rd ed. Wiley — capítulo sobre criptografia aplicada.